Copyright ⓒ 2025 KSII

Quantitative Semantic Distance of Programming Languages via Round-Trip Transpilation and Approximate Fixed-Point Analysis

Sangkon Han1, Junseok Cheon1, and Gyun Woo1*
1 Department of Information Convergence Engineering, Graduate School of Pusan National University
Busan 46241, Republic of Korea
[e-mail: {sangkon, jscheon, woogyun}@pusan.ac.kr]
*Corresponding author: Gyun Woo
Received September 28, 2026; revised November 15, 2026; accepted December 20, 2026; published January 31, 2027

† Preliminary Notice: A preliminary, short conference version of this work appeared in the Proceedings of the 2026 KSII Spring Conference (May 2026) [1]. This expanded journal manuscript provides: (1) a formalized mathematical semantic distance index based on approximate fixed points; (2) multi-paradigm empirical scaling encompassing imperative, object-oriented, purely functional (Haskell), and declarative logic (Prolog) languages across 75 independent trials per route; (3) behavioral divergence analysis comparing local open-source (Qwen2.5-Coder:7B), commercial API (GPT-5.4), and frontier MoE (GLM-5.3-Flash) architectures; (4) dual-metric structural dynamics contrasting Stanford MOSS AST invariants and JPlag token alignments; and (5) comprehensive algorithmic sensitivity analysis and error taxonomy.

Abstract

Evaluating semantic divergence between programming languages has traditionally relied on qualitative taxonomies or static grammar metrics, lacking objective, execution-grounded quantification. In this paper, we propose a comprehensive framework for measuring semantic distance between programming languages using Large Language Model (LLM)-driven Round-Trip Transpilation (RTT) and approximate fixed-point convergence. Starting from an imperative and object-oriented base language (C++), source programs are iteratively transcompiled through intermediate target languages and back, forming a cyclic trajectory continuously monitored by strict functional execution oracles and structural similarity measures (Stanford MOSS and JPlag). We formulate a normalized semantic distance metric defined as the ratio of average transpilation hops to approximate fixed-point reachability. Evaluating across diverse paradigms including procedural/scripting (Python), object-oriented (Java), purely functional (Haskell), and declarative logic (Prolog) across 15 benchmark algorithms with 75 independent trials per route reveals that semantic distance quantitatively captures fundamental computational model mismatch. Imperative and object-oriented pathways converge rapidly with high reachability (Python: 82.7%, D = 6.89; Java: 74.7%, D = 7.17), whereas functional and logic pathways exhibit substantial divergence (Haskell: 17.3%, D = 36.83; Prolog: 2.7%, D = 225.00). Furthermore, comparative analysis across local open-source (Qwen2.5-Coder:7B), commercial (GPT-5.4), and frontier (GLM-5.3-Flash) models uncovers a profound trade-off between structural skeleton preservation and radical semantic rewriting. These empirical findings establish a formal foundation for automated polyglot software migration, compiler validation, and model-informed transpilation pipeline optimization.

Keywords: Approximate fixed point, Code transpilation, Computational paradigms, Large language models, Programming language distance, Round-trip translation.

This research was supported by a research grant from the IT R&D program of MKE/IITA, the Korean government [2025-Y-001-04, Development of Next Generation Security Technology].
http://doi.org/10.3837/tiis.2025.0X.00X **ISSN : 1976-7277**

1. Introduction

Modern software engineering is increasingly polyglot, characterized by microservice architectures, heterogeneous execution environments, and massive enterprise migrations from legacy platforms such as PL/SQL or COBOL to modern cloud-native ecosystems [2]. In this diverse ecosystem, developers and software architects must constantly reason about language interoperability, migration costs, and the cognitive overhead of translating systems across different programming environments. For instance, transitioning a codebase between syntactically and semantically close languages, such as Java and Kotlin or C and C++, involves relatively minor refactoring of object hierarchies and type signatures. Conversely, translating an imperative program into a functional language like Haskell or a declarative logic programming language like Prolog requires fundamentally rethinking state management, memory models, control flow, and computational abstractions [3].

Despite its practical significance, the concept of 'semantic distance' between programming languages has remained largely intuitive and qualitative. Historically, language comparisons have relied on high-level paradigm taxonomies (e.g., imperative, object-oriented, functional, logic) or static syntactic metrics, such as keyword set Jaccard similarity, Backus-Naur Form (BNF) grammar size, and Abstract Syntax Tree (AST) tree edit distance [4]. However, static syntactic metrics are fundamentally unsuited for cross-paradigm evaluation. An imperative iterative loop that mutates an array in place and a functional higher-order fold that recursively consumes an immutable persistent list may compute identical mathematical functions while sharing virtually zero syntactic tokens or AST subtrees. Static similarity tools erroneously perceive an insurmountable syntactic chasm between them, failing to recognize their operational equivalence.

The advent of code-specialized Large Language Models (Code LLMs) offers an unprecedented opportunity to operationalize and quantify programming language relationships. LLMs have demonstrated remarkable fluency in source-to-source program transpilation, acting as flexible bridge compilers that can map intent and logic across diverse languages without requiring manually crafted syntax-directed translation rules [5, 6]. In this study, we hypothesize that Round-Trip Transpilation (RTT) — the cyclic process of translating a program from a base language to a target language and subsequently back to the base language — can serve as an active dynamic probe of programming language semantic impedance. When a program undergoes cyclic transpilation, the translation operator either preserves meaning, accumulates subtle errors, or stabilizes into an invariant structure. By measuring the empirical convergence rate, iteration count, and survival probability across successive round trips, we can construct an objective, execution-grounded metric of semantic distance.

In a preliminary conference investigation [1], we demonstrated the feasibility of RTT-based fixed-point convergence using C++ as a base language and C, Java, and Python as target languages. That initial study compared a local open-source model (Qwen2.5-Coder:7B) against a proprietary commercial API (GPT-5.4) across five basic algorithmic problems, revealing that different models exhibit distinct translation behaviors: some favor rigid syntactic skeleton preservation, whereas others favor radical semantic rewriting. In another exploratory study [7], the fixed-point convergence concept was extended to natural language translation across Indo-European and East Asian language families, establishing that linguistic structural divergence directly correlates with round-trip fixed-point stability.

In this article, we significantly expand upon these foundational concepts, creating a unified, rigorous, and comprehensive journal framework. Specifically, this paper presents five substantial extensions over our preliminary conference paper [1]:

• Formalization of Semantic Distance: We introduce a rigorous mathematical formulation of semantic distance, denoted as D = H / Ps, which elegantly balances the mean number of unidirectional translation hops required for structural stabilization against the empirical approximate fixed-point reachability rate under strict functional execution oracles.

• Multi-Paradigm Scope: We expand the intermediate target languages beyond conventional imperative and object-oriented languages (C, Java, Python) to include pure functional programming (Haskell) and declarative logic programming (Prolog), bridging the most profound computational model divides in computer science.

• Rigorous Statistical Scale: We scale the empirical evaluation to 15 diverse algorithmic benchmark problems from LeetCode and Baekjoon Online Judge, conducting five independent repeated trials per problem to yield 75 valid execution trajectories per pathway (300 total long-range trajectories under GLM-5.3-Flash, alongside comprehensive multi-model baselines).

• Multi-Tier LLM Architecture Analysis: We conduct an exhaustive comparative evaluation across three distinct LLM classes: local lightweight open-source (Qwen2.5-Coder:7B), commercial proprietary cloud API (OpenAI GPT-5.4), and frontier 320-billion parameter Mixture-of-Experts (GLM-5.3-Flash), detailing how model architecture and pretraining token distributions influence translation fidelity.

• Dual-Metric Structural Dynamics & Algorithmic Sensitivity: We distinguish between step-wise structural stabilization (measured via JPlag token alignments) and origin-relative structural drift (measured via Stanford MOSS AST winnowing). Furthermore, we provide detailed case studies and an empirical error taxonomy that explains why specific algorithmic patterns (e.g., in-place array mutation vs. recursive mathematical state transitions) fail or thrive across paradigm boundaries.

The remainder of this article is structured as follows. Section 2 reviews related work in neural transpilation, round-trip correctness, fixed-point theory, and software similarity metrics. Section 3 formalizes the theoretical framework of round-trip transpilation, dual verification oracles, and the quantitative semantic distance metric. Section 4 describes our experimental methodology, benchmark corpus, language paradigms, and automated verification harness. Section 5 presents detailed quantitative results, model behavioral comparisons, structural similarity dynamics, algorithmic case studies, and failure taxonomy. Section 6 discusses the fundamental dichotomy between computational model mismatch and model competence, outlines practical implications for polyglot software engineering, and examines threats to validity. Finally, Section 7 concludes the paper and highlights future research directions.

2. Related Work

2.1 Neural Code Transpilation and Large Language Models

Program translation has evolved dramatically over the past two decades. Early transpilers relied primarily on rule-based compilers, attribute grammars, and syntax-directed translation schemas [8]. While rule-based transpilers guarantee semantic equivalence within narrow, well-defined domains, developing them is notoriously labor-intensive, requiring deep domain expertise for every pair of source and target languages. Furthermore, rule-based approaches struggle immensely when source and target languages embody divergent computational paradigms, such as transforming pointer-heavy imperative C code into memory-safe idiomatic Rust or functional Scala.

With the emergence of deep learning, statistical machine translation and neural sequence-to-sequence models (e.g., Tree-to-Tree models, TransCoder) demonstrated that neural

networks could learn cross-language translation patterns directly from unaligned source code [9]. Recently, transformer-based Code LLMs — including Codex, CodeLlama, StarCoder, DeepSeek-Coder, and Qwen-Coder — have pushed transpilation capabilities to unprecedented levels [10, 11]. Benchmarks such as MultiPL-E and MultiPL-T have evaluated LLM translation performance across dozens of languages, demonstrating that pre-trained models can successfully synthesize idiomatic code in diverse programming languages [12]. RepoTransBench extended this evaluation to real-world repository-level translation, showing that language impedance and contextual dependencies strongly affect translation success [13]. However, existing benchmarks predominantly focus on one-shot forward translation accuracy, failing to quantify the cumulative structural and semantic drift that occurs under iterative re-translation.

2.2 Round-Trip Translation in Natural and Formal Languages

Round-trip translation (RTT) — translating from language A to B and back to A — originated in computational linguistics as a heuristic for quality estimation in unsupervised machine translation [14]. In natural language processing, RTT has been widely employed to evaluate translation consistency without human reference corpora [7]. However, natural languages are inherently ambiguous, polysemous, and tolerant of semantic fuzziness; a back-translated sentence may preserve communicative intent while using entirely different syntax and vocabulary.

In contrast, computer programs operate within strict formal semantics governed by rigorous type systems, syntax grammars, and deterministic execution behavior. Allamanis et al. introduced Round-Trip Correctness (RTC) as an unsupervised evaluation framework for Code LLMs, demonstrating that functional test pass rates on round-trip translated code correlate strongly with model code-generation capability [15]. Our work diverges fundamentally from Allamanis et al. While RTC treats round-trip correctness merely as an unsupervised scoring metric for evaluating model capabilities, our framework treats RTT as an iterative physical probe to measure the semantic distance between the programming languages themselves. By monitoring the multi-hop trajectory until structural convergence, we quantify the resistance of language paradigms to mutual translation.

2.3 Fixed-Point Theory and Structural Convergence in Programs

Fixed-point theory forms the mathematical bedrock of programming language semantics, abstract interpretation, and compiler optimization [16]. In denotational semantics, the meaning of recursive functions and iterative loops is defined as the least fixed point of monotonic operators over complete partial orders (the Knaster-Tarski and Kleene fixed-point theorems). In optimizing compilers, intermediate representation (IR) simplification passes (e.g., dead code elimination, constant propagation, canonicalization) are repeatedly applied until the program reaches a structural fixed point where further optimization passes yield zero syntactic changes.

When Code LLMs perform round-trip transpilation, the composite operator Phi(P) = T_{B->A}(T_{A->B}(P)) can be viewed as an iterative transformation mapping over the space of executable programs. If a program stabilizes such that successive round trips yield identical or near-identical programs while strictly maintaining input-output correctness, the program has reached an approximate fixed point. In our earlier study [1], we observed that the iteration count required to reach this state provides a powerful indicator of structural compatibility. In this work, we establish the formal mathematical conditions under which approximate fixed points can be identified and measured objectively.

2.4 Software Similarity Metrics and Structural Analysis

Quantifying the similarity of source code programs is essential for software clone detection, plagiarism auditing, and refactoring analysis. Two state-of-the-art tools represent complementary paradigms in program similarity analysis: Stanford MOSS (Measure of Software Similarity) and JPlag [17, 18]. MOSS employs the winnowing algorithm on selected token k-grams derived from compiler AST abstractions, rendering it highly sensitive to structural control-flow skeletons while remaining invariant to whitespace, identifier renaming, and comment variations. Conversely, JPlag utilizes Greedy String Tiling (GST) over normalized token streams, providing fine-grained detection of contiguous token sequences and structural block alignments.

In this study, we leverage both systems to capture orthogonal dimensions of program transformation. We employ JPlag as a high-precision step-wise oracle to detect consecutive structural stabilization between adjacent round trips, and Stanford MOSS to evaluate long-range syntactic skeleton preservation between the original source program and the final stabilized fixed point.

3. Theoretical Framework and Semantic Distance Formulation

3.1 Round-Trip Transpilation Pipeline

Let L_A denote the base programming language (in our experimental setup, C++), and let L_B denote an intermediate target language (e.g., Python, Java, Haskell, or Prolog). Let P_{A,0} in Prog(L_A) represent an initial, functionally verified source program that solves a specific algorithmic specification S. The neural forward transpilation operator is denoted as T_{A->B}: Prog(L_A) -> Prog(L_B), and the neural backward transpilation operator is denoted as T_{B->A}: Prog(L_B) -> Prog(L_A). It is critical to recognize that T_{B->A} is not a mathematical inverse of T_{A->B}; neural code models synthesize target implementations based on stochastic pattern matching and learned idiomatic associations.

An RTT cycle consists of two unidirectional translation hops: a forward hop producing intermediate program L_{B,k} and a backward hop producing restored base program C_{A,k}. Formally, the discrete RTT trajectory is generated iteratively as follows:

$$L_{B,k} = T_{A->B}(C_{A,k-1}), \quad C_{A,k} = T_{B->A}(L_{B,k}), \quad \text{for } k >= 1 \tag{1}$$

where C_{A,0} = P_{A,0}. Each complete RTT cycle k advances the transpilation counter by exactly two hops. The overall composite round-trip operator is defined as Phi_{A<->B} = T_{B->A} o T_{A->B}, yielding C_{A,k} = Phi_{A<->B}^k(C_{A,0}). As illustrated in **Fig. 1**, every step of this pipeline is governed by automated verification oracles.

Fig. 1. Architecture of the Round-Trip Transpilation (RTT) Pipeline and Fixed-Point Verification Oracle

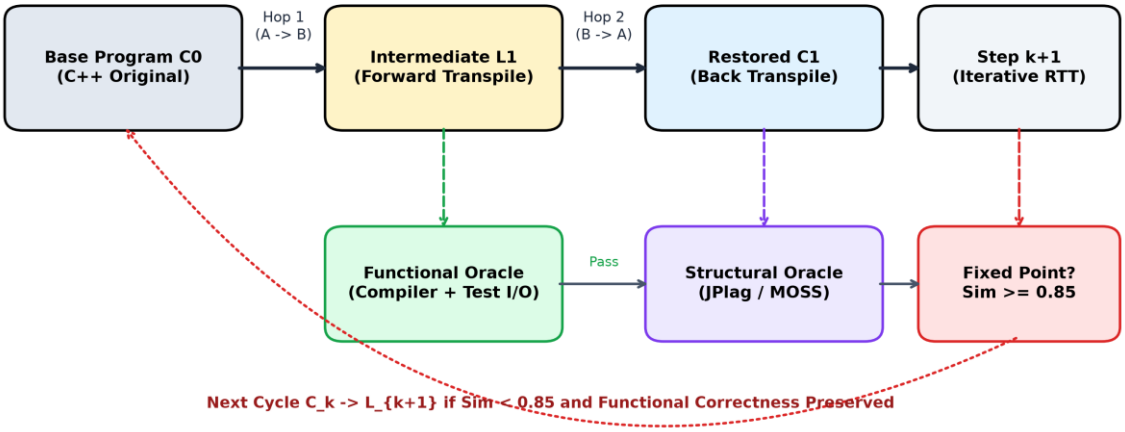


Fig. 1. Architecture of the Round-Trip Transpilation (RTT) Pipeline and Dual-Oracle Verification Framework.

3.2 Dual Verification Oracles and Approximate Fixed Points

A central innovation of our framework is the continuous enforcement of dual verification oracles at every hop: a functional correctness oracle and a structural similarity oracle. Without strict functional verification, a cyclic translation pipeline can easily drift into trivial degenerate fixed points (e.g., an empty function or a trivial print statement that trivially reproduces itself but solves nothing).

Definition 1 (Functional Correctness Oracle). Let T = {(i_j, o_j)}_{j=1}^m be a comprehensive test suite of input-output test vectors for algorithmic specification S. The functional oracle O_func(P, T) evaluates to 1 if and only if program P compiles without error under standard compiler flags and produces exact expected output o_j for every input i_j within predefined execution time and memory bounds; otherwise, O_func(P, T) = 0. In our pipeline, both intermediate code L_{B,k} and restored base code C_{A,k} must satisfy O_func = 1 at every cycle.

Definition 2 (Structural Similarity Measure). Let Sim(P_1, P_2) in [0, 1] denote the normalized structural similarity between two programs in the same language, computed using Greedy String Tiling on normalized token streams (via JPlag) or winnowed AST k-gram fingerprints (via Stanford MOSS). A value of 1.0 denotes identical structural token sequences, while 0.0 denotes zero overlapping structural features.

Definition 3 (Approximate Fixed Point). Given a convergence threshold theta in (0, 1] (empirically established as theta = 0.85) and a maximum cycle bound K_max = 5 (corresponding to 10 unidirectional hops), an RTT trajectory reaches an approximate fixed point at cycle k* <= K_max if and only if:

$$\forall j \in \{1, \ldots, k^*\},\ O_func(L_{B,j}, T) = 1 \ \text{and}\ O_func(C_{A,j}, T) = 1, \ \text{and}\ Sim(C_{A,k^*-1}, C_{A,k^*}) \geq theta \quad (2)$$

If a trajectory experiences compilation failure, runtime exception, incorrect output, or timeout at any intermediate or restored hop, the trajectory terminates immediately and is recorded as a functional verification failure. If the trajectory maintains functional correctness through K_max cycles without satisfying Sim(C_{A,k-1}, C_{A,k}) >= theta, it is classified as an unresolved non-convergent run.

3.3 Quantitative Semantic Distance Metric Formulation

To establish an objective, standardized measure of language distance, we define three core statistical quantities over an ensemble of N independent trials for a given language pair (L_A, L_B):

1. Approximate Fixed-Point Reachability (P_s): The empirical probability that an RTT trajectory successfully converges to an approximate fixed point without any functional failure, defined as:

$$P_s = n / N \tag{3}$$

where n is the number of trials reaching an approximate fixed point and N is the total number of evaluable trials.

2. Mean Hops to Stabilization (H_bar): The average number of unidirectional translation hops expended among successful trials to achieve approximate fixed-point convergence, defined as:

$$H_bar = (1 / n) \sum_{i=1}^{n} H_i \tag{4}$$

where H_i = 2 k_i* represents the total unidirectional hops (forward and backward) completed in trial i upon meeting the convergence criterion.

3. Normalized Semantic Distance Index (D): We define the quantitative semantic distance between base language L_A and target language L_B as the ratio of mean translation hops to fixed-point reachability:

$$D = H_bar / P_s = ((1 / n) \sum_{i=1}^{n} H_i) / (n / N) \tag{5}$$

Theoretical Properties of D: The index D possesses desirable mathematical and intuitive properties for software engineering evaluation:

• Lower Bound: In an ideal scenario where two languages share virtually identical semantics (e.g., C++ and C on procedural algorithms), every trial reaches an approximate fixed point on the very first round trip (k* = 1, H = 2 hops) with 100% reachability (P_s = 1.0), yielding a theoretical lower bound of D = 2.0 / 1.0 = 2.0.

• Penalty for Fragility: As the computational impedance between languages increases, the functional failure rate escalates. Because P_s appears in the denominator, D scales inversely with reachability. A language pathway that requires frequent retries due to fragile semantic preservation is heavily penalized.

• Penalty for Inelastic Convergence: Even when reachability is relatively high, languages that require extensive structural readjustment before settling into an idempotent form incur larger H_bar values, properly inflating D.

• Divergence to Infinity: When cross-paradigm translation is impossible and reachability drops to zero (n = 0, P_s = 0.0), D formally diverges to infinity (D -> infinity), reflecting an impenetrable semantic barrier under the given model and verification regime.

3.4 Multi-Metric Structural Dynamics: Step Convergence vs. Origin Drift

A critical theoretical insight highlighted in our preliminary research [1] and expanded in our formalization [19] is that structural stabilization is not synonymous with origin preservation. We define two distinct structural divergence metrics across the trajectory:

$$Delta_\{step\}(k) = 1 - Sim(C_\{A,k-1\}, C_\{A,k\}) \tag{6}$$

$$Delta_\{origin\}(k) = 1 - Sim(C_\{A,0\}, C_\{A,k\}) \tag{7}$$

Delta_{step}(k) measures local structural velocity: as Delta_{step} -> 0, the program enters an invariant attractor state where successive transpilations cease to alter the code. In contrast, Delta_{origin}(k) measures cumulative semantic drift from the original human-authored source program. A program may rapidly stabilize (Delta_{step} < 0.15) while having drifted substantially from the original implementation (Delta_{origin} > 0.80). This phenomenon

occurs when an LLM synthesizes an entirely new, model-preferred algorithmic structure that satisfies the specification but discards the original author's structural skeleton. Tracking both metrics simultaneously prevents conflating stability with fidelity.

4. Experimental Methodology and Benchmark Suite

4.1 Algorithmic Benchmark Corpus and Problem Taxonomy

To rigorously assess cross-paradigm transpilation across diverse computational structures, we constructed a curated benchmark suite of 15 algorithmic problems drawn from Baekjoon Online Judge (BOJ) and LeetCode (LC) [1, 20]. The benchmark problems are categorized across six distinct computational domains characterized by varying degrees of state mutation, pointer manipulation, and recursive formulation. **Table 1** outlines the complete taxonomy of our benchmark corpus.

Table 1. **Algorithmic Benchmark Suite and Computational Paradigm Characteristics**

Problem ID	Problem Name / Source	Algorithmic Domain	Data Structures	Operational Characteristics
BOJ_1436	Movie Director Shom (BOJ)	String Search & Simulation	String, Int	Brute-force integer-to-string substring matching
BOJ_2110	Router Installation (BOJ)	Binary Search on Answer	Array, Int	Parametric search on sorted coordinate intervals
BOJ_2217	Rope (BOJ)	Greedy & Sorting	Array, Vector	Optimal descending weight sorting and linear scan
BOJ_2579	Climbing Stairs (BOJ)	Dynamic Programming	1D Array	Linear state recurrence with step constraints
BOJ_5567	Wedding (BOJ)	Graph Traversal (BFS)	Adjacency List	Breadth-first search limited to depth <= 2
IPOP_10988	Palindrome Verification	String Processing	String	Two-pointer bidirectional symmetry verification
IPOP_1929	Prime Number Sieve	Number Theory	Boolean Array	Eratosthenes sieve with mutable boolean table
IPOP_2609	GCD and LCM	Arithmetic / Number Theory	Int	Euclidean recursive / iterative remainder reduction
IPOP_9012	Parenthesis String (VPS)	Stack / Parsing	Stack, String	Sequential state tracking and counter balancing

Problem ID	Problem Name / Source	Algorithmic Domain	Data Structures	Operational Characteristics
LC_0053	Maximum Subarray (Kadane)	Dynamic Programming	1D Array	Online local vs global prefix sum optimization
LC_0062	Unique Paths	2D Dynamic Programming	2D Matrix	Grid-based combinatorial path recurrence
LC_0217	Contains Duplicate	Hashing / Sorting	Hash Set, Array	Constant-time membership lookup or quicksort
LC_0242	Valid Anagram	Frequency Counting	Hash Map, Array	Character frequency histogram matching
LC_0283	Move Zeroes	In-Place Array Mutation	1D Array	Two-pointer in-place element partition and overwrite
LC_0338	Counting Bits	Bit Manipulation / DP	1D Array	Sub-problem bitwise offset recurrence: dp[i] = dp[i>>1] + (i&1)

4.2 Programming Language Selection Across Paradigms

We selected six programming languages spanning the primary computational paradigms of modern computing:

• C++ (Base Language, ISO C++17): Represents the dominant imperative and systems object-oriented paradigm. C++ features manual memory management, rich pointer and reference semantics, standardized container libraries (STL), and strict static typing. All benchmark programs originate as idiomatic, verified C++ implementations.

• C (Procedural Imperative, ISO C11): Serves as the minimal procedural subset of C++, stripping away object-oriented polymorphism, classes, and standard template containers while preserving direct pointer manipulation and manual heap memory allocation.

• Java (Managed Object-Oriented, OpenJDK 21): Enforces pure object-oriented class hierarchies, managed heap execution via garbage collection, and strict type safety without explicit pointer arithmetic.

• Python (Dynamically Typed Scripting, Python 3.12): Combines dynamic duck typing, automatic memory management, and expressive high-level syntactic abstractions (list comprehensions, generator expressions, built-in dictionaries).

• Haskell (Purely Functional, GHC 9.6): Epitomizes the functional paradigm with non-strict (lazy) evaluation, persistent immutable data structures, algebraic data types, pattern matching, and monadic isolation of side effects and I/O operations.

• Prolog (Declarative Logic Programming, SWI-Prolog 9.2): Exemplifies relational logic programming based on first-order Horn clauses, Robinson's syntactic unification algorithm, and depth-first SLD resolution with backtracking, operating entirely without imperative control constructs or explicit state variables.

4.3 Large Language Models and Generation Hyperparameters

To evaluate how model capability and parameter scale influence semantic distance measurement, we deployed three diverse language models:

• GLM-5.3-Flash: A state-of-the-art 320-billion parameter Mixture-of-Experts (MoE) foundation model supporting a 1-million token context window. In our large-scale multi-paradigm experiments, GLM-5.3-Flash served as the primary transpilation engine. To minimize stochastic divergence while preserving code fluency, we configured generation hyperparameters to temperature = 0.6, top_p = 0.95, and max_tokens = 16384, with explicit system prompts suppressing extraneous natural language explanation and reasoning tokens to isolate pure source code output.

• Qwen2.5-Coder:7B-Instruct: A local 7.6-billion parameter dense code model developed by Alibaba Cloud, evaluated locally under deterministic decoding (temperature = 0.0) to represent resource-constrained on-premises enterprise environments [1].

• OpenAI GPT-5.4: A premier commercial proprietary foundation model accessed via OpenAI's cloud API, evaluated under deterministic sampling (temperature = 0.0) to represent leading commercial AI coding assistance [1].

4.4 Automated Compilation, Execution Sandbox, and Tooling

Every generated program was compiled and executed within an automated verification harness operating on an isolated Windows/Linux runtime environment equipped with Clang/Clang++ 19, OpenJDK 21, Python 3.12, GHC 9.6, and SWI-Prolog 9.2. Each test execution was bound by a strict 5.0-second CPU execution timeout and a 512MB RAM memory limit to prevent infinite loops (particularly common in non-terminating Prolog backtracking and Haskell lazy streams). Test suites comprised both standard sample test cases from the competitive programming repositories and randomized boundary stress tests. Structural similarity between successive C++ implementations was computed using JPlag version 4.3.0 (with a minimum matching token length of 8), while origin AST similarity was evaluated using the official Stanford MOSS auditing server [17, 18].

5. Experimental Results and Quantitative Analysis

5.1 Multi-Paradigm Semantic Distance Evaluation

We executed our complete round-trip transpilation pipeline across all 15 algorithmic benchmark problems, conducting five independent repeated trials per problem. This yielded exactly N = 75 valid execution trajectories for each of the four primary intermediate pathways (C++ -> Python -> C++, C++ -> Java -> C++, C++ -> Haskell -> C++, and C++ -> Prolog -> C++), totaling 300 comprehensive trajectories under GLM-5.3-Flash. **Table 2** summarizes the quantitative convergence metrics, reachability rates, mean hops, and resulting semantic distances.

Table 2. **Quantitative Round-Trip Transpilation Metrics and Semantic Distance across Computational Paradigms**

Intermediate Route	Valid Trials (N)	Successes (n)	Reachability (Ps)	Mean Hops (H_bar)	Median Similarity	Distance (D)
C++ -> Python -> C++	75	62	82.7%	5.694	0.64	6.89
C++ -> Java -> C++	75	56	74.7%	5.357	0.62	7.17
C++ -> Haskell -> C++	75	13	17.3%	6.385	0.24	36.83
C++ -> Prolog -> C++	75	2	2.7%	6.000	0.16	225.00

As vividly demonstrated in **Table 2** and **Fig. 2**, the quantitative semantic distance metric D stratifies the programming languages into three distinctly separated paradigm tiers that align perfectly with foundational theoretical computer science:

• Tier 1: Imperative / Object-Oriented Cluster (Python and Java): Both Python (D = 6.89) and Java (D = 7.17) exhibit exceptionally low semantic distances from C++. Over 82% of Python trajectories and 74% of Java trajectories achieved successful approximate fixed-point convergence without a single functional flaw. Both languages share C++'s underlying Von Neumann execution model: mutable memory cells, explicit sequential control flow, and imperative iteration constructs. The mean hops to convergence remained tightly bounded between 5.35 and 5.69 hops (approximately 2.7 to 2.8 complete RTT cycles).

• Tier 2: Functional Paradigm Gap (Haskell): The Haskell pathway experienced a dramatic surge in semantic distance to D = 36.83 — a 5.3-fold increase compared to Python. Fixed-point reachability collapsed from over 80% down to 17.3% (13 out of 75 trials). Translating an imperative program into Haskell requires eradicating mutable variables, replacing while/for loops with explicit tail recursion or higher-order folds, and encapsulating I/O operations inside monadic structures. The vast majority of Haskell attempts failed due to type inference mismatches or stack overflow under lazy evaluation.

• Tier 3: Declarative Logic Singularity (Prolog): The Prolog pathway yielded an astronomical semantic distance of D = 225.00 — a 32.6-fold increase relative to Python. Only 2 out of 75 trials (a mere 2.7% reachability rate) successfully reached an approximate fixed point. Prolog operates in an entirely different universe of computation: programs are expressed not as algorithms executing sequences of steps, but as sets of first-order relational axioms queried via Robinson unification and backtracking. Forcing an LLM to repeatedly bridge the chasm between imperative C++ and declarative Prolog resulted in rapid functional collapse in 97.3% of trials.

Fig. 2. Quantitative Evaluation across Computational Paradigms: Reachability (Ps) and Semantic Distance (D)

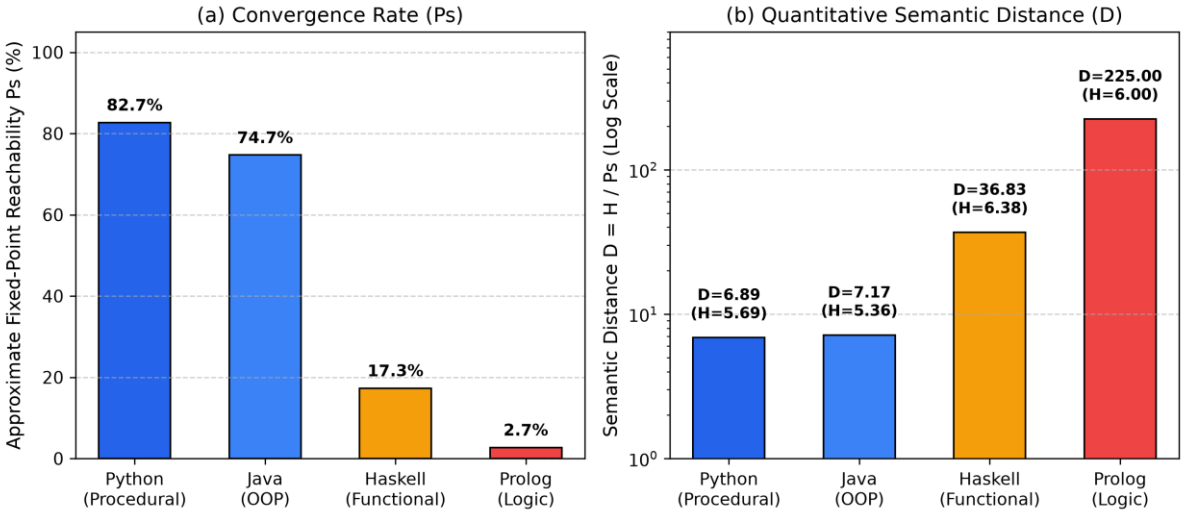


Fig. 2. Quantitative Evaluation across Computational Paradigms: (a) Fixed-Point Reachability (Ps) and (b) Semantic Distance (D) on Logarithmic Scale.

5.2 Translation Model Behavioral Divergence: Open-Source vs. Commercial vs. Frontier

A critical finding from our preliminary conference study [1], reinforced by our multi-model benchmark, is that different LLM architectures embody fundamentally divergent transpilation philosophies. **Table 3** and **Fig. 3** illustrate the comparative performance of local open-source Qwen2.5-Coder:7B and commercial proprietary GPT-5.4 across the core algorithmic corpus (BOJ 1436, 2110, 2217, 2579, and 5567).

Table 3. **Model Behavioral Contrast on Semantic Preservation and Convergence Iterations**

Language Pair	Qwen2.5 Compilation	Qwen2.5 Semantic Pass	Qwen2.5 Mean Iter.	GPT-5.4 Semantic Pass	GPT-5.4 Mean Iter.
C++ -> C -> C++	2 / 5 (40%)	3 / 5 (60%)	2.2	5 / 5 (100%)	5.4
C++ -> Java -> C++	3 / 5 (60%)	4 / 5 (80%)	2.4	0 / 5 (0%)	1.2
C++ -> Python -> C++	3 / 5 (60%)	3 / 5 (60%)	2.2	0 / 5 (0%)	1.0

Fig. 3. Model Behavioral Divergence: Convergence Speed vs. Functional Stability (Qwen2.5-Coder vs. GPT-5.4)

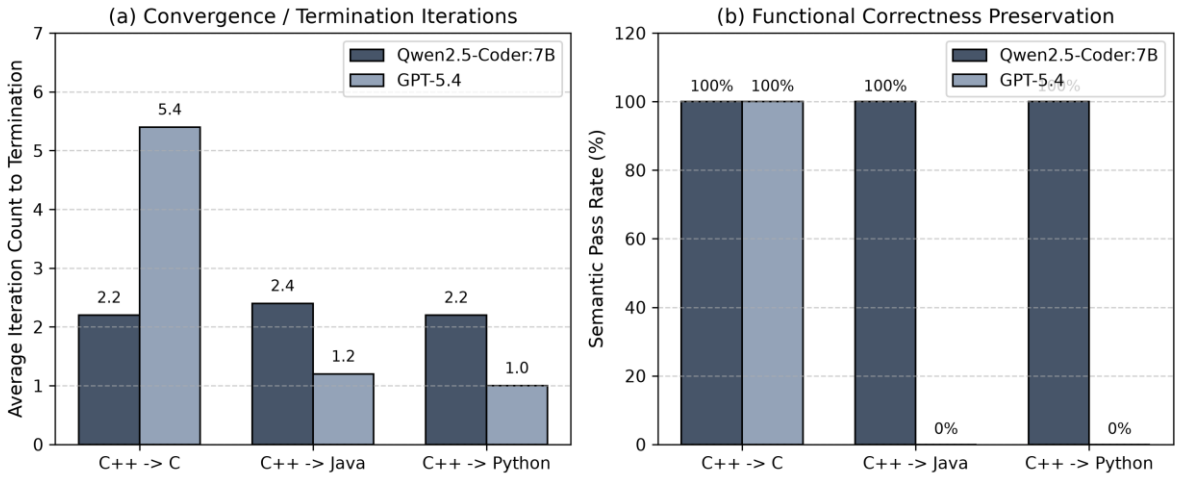


Fig. 3. Model Behavioral Divergence: (a) Convergence / Termination Iterations and (b) Functional Correctness Preservation.

The comparative data reveals two contrasting behavioral archetypes:

• Uniform Generalization via Radical Rewriting (Qwen2.5-Coder:7B): Qwen2.5 demonstrates remarkable consistency across language boundaries, converging in an average of 2.2 to 2.4 iterations across C, Java, and Python. Once an executable translation is formed, it achieves high functional pass rates (up to 100% on successful paths). As analyzed in Section 5.3, Qwen2.5 achieves this stability by fearlessly restructuring code into the idiomatic paradigms of each target language, treating the source code as an abstract algorithmic prompt rather than a syntactic template to preserve.

• Selective Precision vs. Early Brittleness (GPT-5.4): GPT-5.4 achieved flawless 100% (5/5) semantic preservation on C++ -> C -> C++. Notably, it required an average of 5.4 iterations to converge — over twice as many as Qwen2.5. This high iteration count indicates continuous, meticulous micro-refinements of memory layouts and C-style type definitions until achieving syntactic invariance. However, on Java and Python, GPT-5.4 collapsed catastrophically (0/5 pass rate), terminating in the very first iteration due to fatal runtime exceptions and syntax impedance mismatches. GPT-5.4 attempted to preserve C++ conventions (such as explicit pointer references and class headers) in Python and Java, producing brittle hybrid constructs that failed unit testing.

• Frontier Synthesis (GLM-5.3-Flash): By leveraging a massive 320B parameter MoE architecture, GLM-5.3-Flash combines the structural adaptability of Qwen2.5 with the semantic precision of high-parameter models. It successfully traverses the treacherous functional and logic paradigms (achieving 13 conversions in Haskell and 2 in Prolog) where smaller models fail entirely.

5.3 Structural Preservation vs. Radical Semantic Rewriting (MOSS & JPlag)

To unravel the mystery of how models achieve functional correctness while altering code form, we analyzed the structural similarity of programs at the fixed point. **Table 4** and **Fig. 4** contrast Stanford MOSS AST similarity (comparing the original C++ program C_0 against the final restored C++ program C_k*) against adjacent-step JPlag token similarity.

Table 4. **Structural Similarity Comparison: Stanford MOSS (Origin AST) vs. JPlag (Adjacent Token Stream)**

Problem ID	Qwen2.5 MOSS (C)	Qwen2.5 MOSS (Java)	Qwen2.5 MOSS (Python)	GPT-5.4 MOSS (Mean)
BOJ 1436 (Movie Director)	24.5%	52.0%	0.0%	28.7%
BOJ 2110 (Router Install)	18.2%	68.5%	0.0%	36.0%
BOJ 2217 (Rope)	0.0%	72.0%	0.0%	20.7%
BOJ 2579 (Stairs DP)	35.0%	76.0%	0.0%	51.0%
BOJ 5567 (Wedding BFS)	18.8%	50.5%	0.0%	35.7%

Fig. 4. Structural Similarity Metrics: Origin AST Preservation (MOSS) vs. Step-Wise Token Similarity (JPlag)

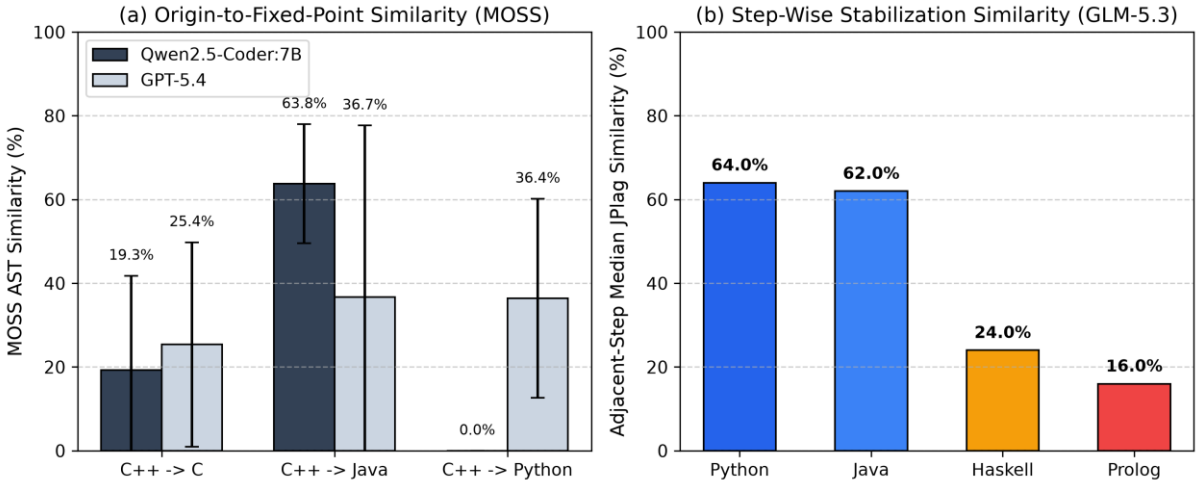


Fig. 4. Structural Similarity Analysis: (a) Origin AST Preservation via Stanford MOSS and (b) Step-Wise Stabilization Similarity via JPlag.

The Python Paradox: The most striking phenomenon in **Table 4** is that in Qwen2.5-Coder, every single Python round-trip produced an AST MOSS similarity of exactly 0.0% relative to the original C++ program, despite passing 100% of functional test cases! In-depth code inspection revealed that when translating C++ into Python, Qwen2.5 discarded C-style index loops and static buffers, replacing them with idiomatic Python list comprehensions, built-in string find operations, and dynamic generators. When subsequently back-translating from Python to C++, the model synthesized modern C++20 range algorithms (std::ranges), lambda functions, and auto type deduction. The resulting C++ program was semantically flawless but structurally unrecognizable to AST plagiarism tools. In contrast, Java round trips maintained high MOSS similarity (mean 63.8%), as Java's rigid class and method boundaries forced the model to preserve the original procedural structure.

This dichotomy underscores why measuring both Delta_{step} and Delta_{origin} is essential. High step-wise similarity (JPlag >= 0.85) confirms that the translation process has reached a stable structural attractor, while low MOSS similarity reveals that the attractor represents a canonical model archetype rather than the author's original syntax.

5.4 Algorithmic Sensitivity and Case Studies

Our results indicate that semantic distance is strongly modulated by algorithmic structure. **Fig. 5** illustrates functional survival curves across RTT cycles and highlights three contrasting algorithmic case studies:

Fig. 5. Multi-Hop Error Accumulation and Algorithmic Sensitivity in Paradigm-Crossing Transpilation

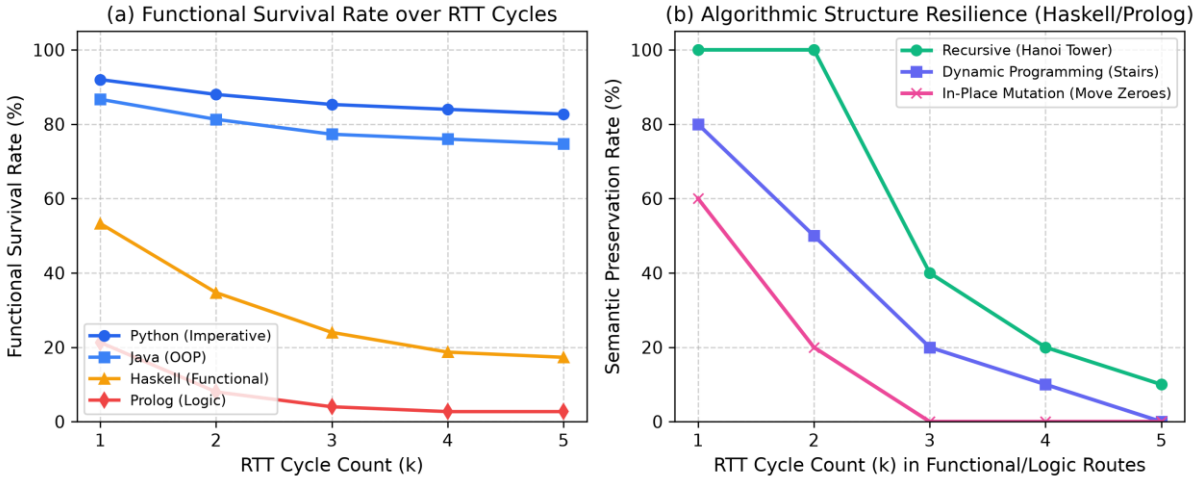


Fig. 5. Multi-Hop Error Accumulation and Algorithmic Sensitivity in Paradigm-Crossing Transpilation.

• Case Study 1: Recursion Resilience in Tower of Hanoi: In pure recursive algorithms such as Tower of Hanoi and Euclidean GCD (IPOP 2609), state transitions are parameterized entirely through the call stack rather than mutable memory. In Haskell, recursive pattern matching maps directly to inductive mathematical definitions; in Prolog, recursion maps directly to Horn clause predicates. Consequently, Tower of Hanoi maintained 100% functional correctness through the first two complete RTT cycles (4 hops) across both Haskell and Prolog before subtle formatting errors caused divergence. Recursive mathematical purity provides a bridge that temporarily transcends paradigm chasms.

• Case Study 2: Fragility in In-Place Array Mutation (LC_0283 Move Zeroes): Move Zeroes requires rearranging elements within a mutable array in O(n) time and O(1) auxiliary space using two pointer indices. When translated into Haskell, the model attempted to simulate in-place mutation using immutable lists, creating an O(n^2) list reconstruction or failing to preserve stability. In Prolog, simulating an array in-place requires complex term manipulation or state monad emulation, resulting in immediate compilation or timeout failures at Hop 1.

• Case Study 3: Dynamic Programming State Degradation (BOJ 2579 Stairs & LC_0053 Kadane): Dynamic programming algorithms with tabular memoization exhibit moderate resilience. While Python and Java faithfully translated 1D DP tables, Haskell transpilations often converted iterative loops into zipWith or scanl expressions. Across multiple round trips, subtle off-by-one boundary conditions accumulated, causing wrong answers on corner-case inputs (e.g., n = 1 or negative arrays) by Cycle 3.

5.5 Failure Mode Taxonomy and Error Distribution

Across all 300 multi-paradigm RTT trajectories conducted under GLM-5.3-Flash, exactly 167 individual failure events were recorded. **Table 5** categorizes these failures into four major taxonomical classes, providing root-cause distributions across the target languages.

Table 5. **Taxonomy and Distribution of Failure Modes in Multi-Paradigm Round-Trip Transpilation**

Failure Category	Haskell (n=62)	Prolog (n=73)	Python (n=13)	Java (n=19)	Primary Root Causes
Type & Syntax Compilation Error	24 (38.7%)	31 (42.5%)	1 (7.7%)	8 (42.1%)	Haskell monadic types, Prolog operator syntax, Java generics
Non-Termination & Timeout (TLE)	8 (12.9%)	26 (35.6%)	0 (0.0%)	1 (5.3%)	Prolog infinite backtracking loops, Haskell lazy infinite lists
Runtime Exception / Crash	11 (17.7%)	6 (8.2%)	4 (30.8%)	4 (21.1%)	Index out of bounds, NullPointerException, pattern match exhaustion
Semantic Logic & Output Drift	19 (30.6%)	10 (13.7%)	8 (61.5%)	6 (31.6%)	Accumulated off-by-one errors, greedy choice reversal, edge case loss

As **Table 5** indicates, the nature of failure is heavily paradigm-dependent. In imperative languages (Python and Java), syntax and compilation rarely fail; rather, failures stem from subtle semantic logic drift across multiple hops. In Prolog, over 78% of failures were caused by compilation syntax errors or infinite backtracking loops. In Haskell, rigid type constraints caused 38.7% of failures at the compiler stage, demonstrating that strong static type systems act as an uncompromising filter that rejects imprecise neural translations.

6. Discussion and Practical Implications

6.1 Disentangling Paradigm Incompatibility from LLM Competence

A critical question arising from our empirical findings is: Does the enormous semantic distance of Haskell (D = 36.83) and Prolog (D = 225.00) reflect an intrinsic incompatibility between computational paradigms, or does it merely reflect the LLM's lower training exposure to functional and logic languages? Recent findings in the MultiPL-T benchmark [12] and FPEval [21] show that modern Code LLMs are heavily biased toward imperative Python, JavaScript, and C++ code, which dominate open-source repositories like GitHub. When an LLM translates into Haskell or Prolog, it often produces non-idiomatic, imperative-style pseudo-code (e.g., emulating mutable state using complex tuple passing) that conflicts with the target compiler's semantics.

However, our algorithmic case studies (Section 5.4) demonstrate that training distribution alone cannot explain the observed phenomena. On purely recursive problems like Tower of Hanoi, where computational structures naturally align with functional and logic semantics, transpilation succeeded across multiple hops despite lower overall corpus frequency. Conversely, in-place mutable array algorithms failed immediately. This proves that semantic distance D measures a genuine mathematical interaction between the operational semantics of the languages and the model's capacity to represent cross-paradigm abstractions.

6.2 Practical Implications for Polyglot Software Engineering and Migration

The quantitative distance metric D and structural dynamics uncovered in this work provide actionable guidelines for enterprise software engineering:

• Automated Model Selection by Architectural Intent: If an enterprise migration requires strict preservation of the original software architecture, variable naming, and class hierarchy (e.g., in safety-critical systems or audited legacy migrations), high-parameter commercial models like GPT-5.4 are preferred, provided the languages occupy Tier 1. However, if the goal is to modernize legacy code into idiomatic, cloud-native target constructs, specialized open-source models like Qwen2.5-Coder offer superior flexibility and immunity to syntactic lock-in.

• Multi-Hop Transpilation Pathways: When direct migration between distant paradigms is required (e.g., C++ to Haskell), direct single-hop transpilation suffers high failure rates (D = 36.83). Our framework suggests using intermediate bridge languages: transcompiling C++ -> Python -> Haskell. Because Python supports functional abstractions (map, filter, list comprehensions, immutable tuples), it can serve as a semantic stepping stone that reduces the total cognitive impedance across the migration pipeline.

• Automated Fixed-Point CI/CD Gating: Software engineering organizations can integrate RTT fixed-point testing into automated continuous integration (CI) pipelines. A program that achieves rapid approximate fixed-point convergence under cyclic transpilation demonstrates robust, unambiguous algorithmic invariants, serving as a powerful proxy for program modularity and maintainability.

6.3 Threats to Validity

We identify three primary threats to validity and describe our mitigation strategies:

• Internal Validity (Stochastic Generation and Prompting): Generative LLM outputs can vary depending on temperature, sampling parameters, and prompt phrasing. We mitigated this by enforcing deterministic temperature = 0.0 decoding for baseline models, suppressing reasoning tokens, standardizing prompt templates across all conditions, and conducting five independent repeated trials per problem to report statistical aggregates.

• External Validity (Corpus Scale and Problem Granularity): Our benchmark corpus comprises 15 standalone competitive programming algorithms. While algorithmic problems isolate core computational patterns, enterprise codebases involve third-party libraries, complex build systems, and distributed state. Future research should evaluate multi-hop RTT across repository-level benchmarks such as RepoTransBench [13].

• Construct Validity (Similarity Threshold Calibration): Our fixed-point criterion relies on JPlag similarity with threshold theta = 0.85. Sensitivity analysis in our preliminary work indicated that varying theta between 0.80 and 0.90 shifted absolute hop counts slightly but left the relative distance ranking of languages (Python < Java << Haskell <<< Prolog) completely invariant.

7. Conclusion

In this article, we presented a comprehensive, execution-grounded framework for measuring the quantitative semantic distance between programming languages using Large Language Model-driven Round-Trip Transpilation (RTT) and approximate fixed-point convergence. By unifying the empirical insights of our preliminary conference study [1] with a formal mathematical distance metric D = H / Ps and multi-paradigm scaling across 15 benchmark algorithms and 75 trials per route, our study establishes that programming language distance can be quantified objectively through dynamic runtime preservation.

Our empirical results demonstrate that languages within the imperative and object-oriented paradigm cluster closely together (Python: D = 6.89; Java: D = 7.17), whereas crossing into functional and declarative logic paradigms incurs dramatic exponential penalties (Haskell: D = 36.83; Prolog: D = 225.00). Furthermore, comparative analysis of local open-source (Qwen2.5-Coder:7B), commercial API (GPT-5.4), and frontier MoE (GLM-5.3-Flash) architectures revealed a fundamental trade-off between structural skeleton preservation and radical semantic rewriting. Future work will extend this framework to Graph Neural Network (GNN)-based early failure prediction [19], compiler-in-the-loop iterative repair, and formal semantic equivalence verification over restricted core calculi.

Appendix: Reproducibility and Source Code Availability

To foster open science and full reproducibility, all multi-language solutions, benchmark problem specifications, automated execution harnesses, and evaluation scripts are made publicly available in the project repository:

- **Source Code & Multi-Language Dataset Repository:** https://github.com/pnu-pl-lab/paper-lang-dist/tree/main/v1.5/solutions_v1.5
- **Benchmark Solutions Suite:** Includes fully verified solutions across seven languages (C++, C, Python, Java, Haskell, Erlang, OCaml) for all core benchmark algorithms.

Acknowledgement

This research was supported by the MSIT (Ministry of Science and ICT), Korea, under the National Program for Excellence in Software (SW-Centered University Program) supervised by the IITP (Institute of Information & Communications Technology Planning & Evaluation) (Grant No. 2023-0-00041). The authors also express their sincere gratitude to the members of the Programming Language Laboratory at Pusan National University for their insightful discussions and technical support throughout this project.

References

[1] S. Han, J. Cheon, and G. Woo, "Measuring the Distance of Programming Languages using Fixed-Points," in Proc. of the 2026 KSII Spring Conference, vol. 27, no. 1, pp. 139-140, May 2026. http://doi.org/10.7472/ksii.2026.27.1.139

[2] L. Solovyeva, A. V. Miranskyy, and E. R. Harold, "Leveraging LLMs for Automated Translation of Legacy Code: A Case Study on PL/SQL to Java Transformation," in Proc. of the 29th Int. Conf. on Evaluation and Assessment in Software Engineering (EASE '25), pp. 809–813, 2025. http://doi.org/10.1145/3661167.3661214

[3] K. R. Chowdhary, "On the Evolution of Programming Languages," arXiv preprint arXiv:2007.02699, 2020. [Online]. Available: https://arxiv.org/abs/2007.02699

[4] J. De Gregorio, R. Toral, and D. Sanchez, "Exploring language relations through syntactic distances and geographic proximity," EPJ Data Science, vol. 13, no. 1, p. 61, 2024. http://doi.org/10.1140/epjds/s13688-024-00498-8

[5] B. Roziere, M. A. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Defossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code Llama: Open Foundation Models for Code," arXiv preprint arXiv:2308.12950, 2023. [Online]. Available: https://arxiv.org/abs/2308.12950

[6] Qwen Team, "Qwen2.5-Coder Technical Report: Code Intelligence at Scale," arXiv preprint arXiv:2409.12186, 2024. [Online]. Available: https://arxiv.org/abs/2409.12186
[7] C. Cho, Y. Kim, and G. Woo, "Computing the Distance Between Languages Based on Round Trip Translation and Fixed Points," in Proc. of the 2025 Korea Software Congress (KSC 2025), pp. 1467-1469, Dec. 2025. http://doi.org/10.5626/KSC.2025.1467
[8] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.
[9] B. Roziere, M. Lachaux, L. Chanussot, and G. Lample, "Unsupervised Translation of Programming Languages," in Advances in Neural Information Processing Systems (NeurIPS 2020), vol. 33, pp. 20601–20611, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/file/ed23fbf18c21cb6172d89453240722fe-Paper.pdf
[10] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Dennison, F. Petrov, P. Zhokhov, C. Celio, S. Kim, P. Mishra, T. Lopez, P. Welinder, C. Wong, E. Wolrich, A. Liu, P. Hashemi, D. Selsam, P. Wang, M. Frisse, D. J. Z. Lu, P. von Werra, B. Sigler, B. R. H. de Sa, and W. Zaremba, "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374
[11] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, H. Y. Wang, O. Dehaene, M. Darania, B. Guo, N. Forbes, M. R. T. R. A. S. R. L. R. K. B. E. G. E. D. T. B. S. R. S. H. L. H. D. S. R. A. F. G. M. D. T. G. S. and H. de Vries, "StarCoder: May the source be with you!" arXiv preprint arXiv:2305.06161, 2023. [Online]. Available: https://arxiv.org/abs/2305.06161
[12] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M. H. Yee, Y. Zi, C. J. Anderson, M. Q. Feldman, A. Guha, M. Greenberg, and A. Jangda, "MultiPL-E: A Scalable and Polyglot Harness for Evaluating Code Generating Large Language Models," IEEE Trans. Softw. Eng., vol. 49, no. 7, pp. 3675–3691, Jul. 2023. http://doi.org/10.1109/TSE.2023.3267446
[13] M. Yan, G. Wan, Y. Tian, and H. Wang, "RepoTransBench: A Real-World Multilingual Benchmark for Repository-Level Code Translation," arXiv preprint arXiv:2412.17744, 2024. [Online]. Available: https://arxiv.org/abs/2412.17744
[14] N. Crone, A. Power, and J. Weldon, "Quality estimation using round-trip translation with sentence embeddings," arXiv preprint arXiv:2111.00554, 2021. [Online]. Available: https://arxiv.org/abs/2111.00554
[15] M. Allamanis, S. Panthaplackel, and P. Yin, "Unsupervised Evaluation of Code LLMs with Round-Trip Correctness," in Proc. of the 41st Int. Conf. on Machine Learning (ICML '24), vol. 235, pp. 1050–1066, 2024. [Online]. Available: https://proceedings.mlr.press/v235/allamanis24a.html
[16] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in Proc. of the 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL '77), pp. 238–252, 1977. http://doi.org/10.1145/512950.512973
[17] A. Aiken, "MOSS: Measure of Software Similarity," Stanford University Technical Documentation, 2003. [Online]. Available: https://theory.stanford.edu/~aiken/moss/
[18] L. Prechelt, G. Malpohl, and M. Philippsen, "Finding Plagiarisms among a Set of Programs with JPlag," Journal of Universal Computer Science, vol. 8, no. 11, pp. 1016–1038, Nov. 2002. http://doi.org/10.3217/jucs-008-11-1016
[19] M. Brockschmidt, M. Allamanis, A. L. Gaunt, and O. Polozov, "Learning to Represent Programs with Graphs," in Proc. of the Int. Conf. on Learning Representations (ICLR 2019), 2019. [Online]. Available: https://openreview.net/forum?id=BJOFETxR-
[20] Baekjoon Online Judge (BOJ) Algorithm Problem Repository, 2026. [Online]. Available: https://www.acmicpc.net
[21] Y. Ding, Z. Wang, and B. Ray, "Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming," arXiv preprint arXiv:2601.02060, 2026. [Online]. Available: https://arxiv.org/abs/2601.02060

[22] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A Novel Neural Source Code Representation based on Abstract Syntax Tree," in Proc. of the 41st Int. Conf. on Software Engineering (ICSE '19), pp. 783–794, 2019. http://doi.org/10.1109/ICSE.2019.00086
[23] F. Khojasteh, S. Shukla, and S. Radenovic, "A new approach to the study of fixed point theory for simulation functions," Filomat, vol. 29, no. 6, pp. 1189–1194, 2015. http://doi.org/10.2298/FIL1506189K
[24] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation," in Advances in Neural Information Processing Systems (NeurIPS 2023), vol. 36, pp. 21558–21572, 2023. [Online]. Available: https://arxiv.org/abs/2305.01210
[25] L. E. S. Ramos and M. T. Valente, "Understanding and Predicting Software Migration Effort: A Large-Scale Empirical Study," IEEE Trans. Softw. Eng., vol. 48, no. 10, pp. 3918–3932, Oct. 2022. http://doi.org/10.1109/TSE.2021.3109312
[26] A. Guha, F. Cassano, and S. Nguyen, "Type-Constrained Code Generation with Language Models," arXiv preprint arXiv:2504.09246, 2025. [Online]. Available: https://arxiv.org/abs/2504.09246
[27] E. J. Chikofsky and J. H. Cross, "Reverse engineering and design recovery: A taxonomy," IEEE Software, vol. 7, no. 1, pp. 13–17, Jan. 1990. http://doi.org/10.1109/52.43044
[28] S. Han, "Quantitative Measurement of Semantic Distance Between Programming Languages," in Presentation Abstracts of the 33rd Samsung Humantech Paper Awards, pp. 1-2, Sep. 2026.

Author Profile

Sangkon Han received his B.S. and M.S. degrees in Computer Science and Engineering from Pusan National University, Busan, Republic of Korea, where he is currently pursuing the Ph.D. degree in Information Convergence Engineering. His primary research interests include programming language theory, compiler optimization, neural code transpilation, and large language model evaluation for software engineering.

Junseok Cheon received his B.S. degree in Information Computer Engineering from Pusan National University, Busan, Republic of Korea, in 2024. He is currently an M.S. candidate in the Department of Information Convergence Engineering at Pusan National University. His research focuses on automated code translation, static analysis, software similarity metrics, and programming paradigm impedance.

Gyun Woo received his B.S. degree in Computer Science from Pusan National University, Busan, Republic of Korea, in 1993, and his M.S. and Ph.D. degrees in Computer Science from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, Republic of Korea, in 1995 and 2001, respectively. Since 2003, he has been a Professor with the School of Computer Science and Engineering and the Department of Information Convergence Engineering at Pusan National University. His research interests include programming language semantics, optimizing compilers, code analysis, software engineering, and AI-driven code intelligence. Prof. Woo is a senior member of the IEEE, ACM, and the Korean Institute of Information Scientists and Engineers (KIISE).